

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper’s Main Contributions

- C_1 We provide a large-scale, trace-grounded characterization of AI network sensitivity across realistic workloads, fabrics, and scales up to 4,096 GPUs. The study separates the effects of latency, bandwidth, and jitter and identifies the operating regimes in which each limits performance.
- C_2 We develop and validate a compositional linear-programming method with signature-based caching. The method preserves workload dependencies while making production-scale sensitivity studies practical.
- C_3 We derive latency-bandwidth performance envelopes and use them to identify safe operating regions, performance cliffs, and useful network provisioning points.

B. Computational Artifacts

The paper is accompanied by two artifacts. Artifact A_1 contains the analysis code, compact result data, reproduction drivers, plotting scripts, manifests, and the final plots. Artifact A_2 is the companion collection of execution traces used as inputs to the analysis.

- A_1 https://github.com/tommasobo/SC_Tracing (branch artifact_freeze)
- A_2 <http://storage2.spcl.ethz.ch/traces/ai/>

The authors intend to request the Artifacts Available, Artifacts Evaluated-Functional, and Results Reproduced badges. Persistent DOIs will be assigned at the SC26 artifact-freeze stage. No DOI has been assigned at the time of this submission.

Artifact	Contributions	Related paper elements
A_1	C_1, C_2, C_3	Figs. 1, 3–10
A_2	C_1, C_2	Table II, Figs. 3–6

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains the following components:

- solver/: Composite-LP and Monolithic-LP analysis.
- tools/: the NCCL-to-GOAL conversion code and the LogGOPSIm source.
- pipeline/: drivers for trace conversion, LP analysis, LogGOPSIm replay, and selected sensitivity sweeps.
- data/ and results/: compact inputs and outputs for the figure workflows.
- scripts/: one plotting workflow for each reproducible paper figure.

- local_artifact/: provenance manifests and checks for the recovered Figure 5 and Figure 6 inputs.
- reproduce_quick.sh and reproduce_full.sh: the reviewer entry points.

The trace-to-analysis path supports C_1 and C_2 . Its outputs are the sensitivity envelopes, validation comparisons, memory measurements, and jitter results that support C_3 . Figure 2 is a method diagram and has no computational output.

Expected Results

The quick workflow redraws Figures 1 and 3–10 from the compact data. It should complete in a few minutes on a laptop and produce PDF and PNG files under figures/. The layout, curves, relative ordering, and main breakpoints should agree closely with the paper.

The full workflow additionally exercises affordable analysis and trace-processing paths. It supports local execution by default and Slurm execution on Alps or a similar cluster. Long or high-memory stages are disabled unless the user supplies `-expensive_run`. In particular, the Grok 4,096-GPU experiment is not run by default.

The expected scientific results are:

- Composite-LP and the validation baselines have close runtime predictions for the configurations where the baselines are feasible.
- The workload curves have different sensitivity regimes. Llama training is bandwidth-sensitive over a substantial range, Grok at large scale has a broad compute-dominated region, and vLLM inference has a broad memory-dominated region.
- The full-trace Monolithic-LP memory requirement grows quickly with scale, while the compositional method remains practical.
- Sparse jitter has little effect on the studied Grok workload, while perturbing a large fraction of messages produces a clear slowdown.

The repository includes a reproduction report with quantitative comparisons and known differences. The most important limitations are also stated here. The released 128-GPU Llama source is named Llama7B_N32_GPU128, although the paper labels the panel as Llama 70B. The matching scale and the close regenerated curve indicate a paper-label error. The closest recovered Figure 5 Composite curve is raw-derived and differs from the paper CSV by at most 0.014853%. Cold runs from the recovered Llama N4/GPU16 traces do not match it as closely. The recovered Figure 6 vLLM raw trace follows the complete conversion and replay path, but a paper-era token-window reduction step is not available. The default vLLM plot therefore uses the compact paper data, and the report records the raw-replay difference.

Expected Reproduction Time (in Minutes)

Times below are estimates for an ordinary workstation unless a cluster is stated.

- Artifact setup: 5–10 min for cloning, creating a Python environment, and installing the plotting dependencies.
- Quick execution: less than 5 min for `./reproduce_quick.sh`.
- Quick analysis: 10–20 min to run the checks and compare the generated plots with the paper and reference files.
- Full execution without expensive stages: up to 6 h. Each selected figure workflow is designed to finish within about one hour. Independent tasks may be submitted concurrently on Slurm.
- Full analysis: about 30 min for manifest checks, plot inspection, and review of the generated report.

The quick path is sufficient to exercise the packaged artifact. The non-expensive full path fits within the SC26 eight-hour review budget. Grok at 4,096 GPUs and any stage expected to exceed 45 min or require substantial memory remain opt-in and are not part of the recommended reviewer run.

Artifact Setup (incl. Inputs)

Hardware: The quick workflow requires a laptop or workstation with about 1 GiB of free RAM and 1 GiB of free disk space. It does not require a GPU.

The full local workflow should use scratch storage and a host with at least 16 GiB of RAM. Raw NSYS conversion and larger GOAL files may require more memory and disk. The Slurm path was prepared for the CSCS Alps system, whose compute nodes use NVIDIA GH200 superchips and the Slingshot-11 interconnect. Large trace runs must use compute nodes rather than login nodes.

The opt-in Grok 4,096-GPU path and large Monolithic-LP cases can require hundreds of GiB to multiple TiB of memory or very large temporary files. Reviewers should not enable this path without an appropriate allocation.

Software: The default artifact requires:

- Python 3.10 or later, <https://www.python.org/>.
- NumPy 1.21 or later, <https://numpy.org/>.
- pandas 1.3 or later, <https://pandas.pydata.org/>.
- Matplotlib 3.5 or later, <https://matplotlib.org/>.
- Bash 4.4 or later, <https://www.gnu.org/software/bash/>.

The optional LogGOPSIm build requires a C++ compiler, `gengetopt`, and `re2c`: <https://gcc.gnu.org/>, <https://www.gnu.org/software/gengetopt/>, and <https://re2c.org/>.

LP regeneration requires Gurobi 10.0 or later, <https://www.gurobi.com/>. A free academic license is sufficient. Slurm execution requires an installed Slurm client, <https://slurm.schedmd.com/>. Conversion from raw `.nsys-rep` inputs requires NVIDIA Nsight Systems with SQLite export support, <https://developer.nvidia.com/nsight-systems>.

Datasets / Inputs: The compact CSV and JSON inputs required for the default workflows are stored in the repository. They include sensitivity sweeps, hardware reference points, memory results, jitter results, and provenance metadata. These files are small enough for a normal Git checkout.

Artifact A_2 provides larger GOAL, BIN, and NSYS inputs. A reviewer does not need the complete collection. Selected trace directories can be downloaded separately. Raw and intermediate files can be several GiB, while the largest Grok case produces temporary data on the TiB scale.

Figures 1, 7, 8, 9, and 10 use existing compact data by design. The default Figure 5 plot uses the closest recovered raw-derived Composite curve together with existing Monolithic and LGS data. The Grok and vLLM portions of Figure 6 use compact paper data because of the limitations described under Expected Results. All plotting scripts are included.

Installation and Deployment:

```
git clone -b artifact_freeze \
  https://github.com/tommasobo/SC_Tracing.git
cd SC_Tracing
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

No compilation is needed for the quick workflow. The optional LogGOPSIm build is performed by `pipeline/build_tools.sh`. On a Slurm cluster, clone or copy the repository to scratch and keep raw and temporary files there. The full workflow also requires:

```
python3 -m pip install -r requirements-dev.txt
```

This includes the solver and test dependencies. A configured Gurobi license is required for fresh LP sweeps.

Artifact Execution

The workflow consists of five tasks:

- T_1 : validate compact inputs and provenance manifests.
- T_2 : choose the quick plot-only path or the full path.
- T_3 : for enabled full-path experiments, convert or load traces, build dependency metadata, and run Composite-LP, Monolithic-LP, or LogGOPSIm as required.
- T_4 : transform result tables and generate the paper plots.
- T_5 : check the expected output set and write the reproduction report.

The quick dependency chain is $T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_5$. The full dependency chain is $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$. Independent Figure 3–5 tasks may run in parallel. The two Figure 6 solver sweeps are serialized by the Slurm launcher to stay within the available concurrent Gurobi license slots.

The compact-data path is deterministic and needs one run. Hardware reference data and stochastic experiments retain the repetitions used for the paper in the packaged tables. Fresh stochastic or hardware runs must retain their seed and run count in their output metadata.

The two top-level commands are:

```
./reproduce_quick.sh
./reproduce_full.sh
```

The full script runs locally by default. Useful options are `-scratch DIR`, `-workers N`, `-slurm`, and `-dry-run`. The `-expensive_run` flag is the only way to enable Grok 4k and other long or high-memory stages.

Artifact Analysis (incl. Outputs)

Successful execution produces plots under `figures/` and processed tables, logs, and execution manifests under the selected scratch directory. The final reproduction report is under `docs/`. The manifest and artifact checks must pass. The evaluator should compare the generated PDFs with the paper or the supplied side-by-side reference.

The comparison should focus on curve shape, method ordering, breakpoints, and the conclusions listed under Expected Results. Exact pixel equality is not required. Any remaining numeric difference, including the known Figure 5 and vLLM Figure 6 cases, must be recorded in the final report rather than hidden by substituting another workload.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 contains the trace inputs used by the analysis. It supports C_1 by making the communication workloads inspectable and reusable. It supports C_2 by providing GOAL schedules that can be replayed by LogGOPSim and analyzed by the code in A_1 .

Expected Results

A downloaded GOAL trace should parse and replay successfully with the tools in A_1 . A selected moderate-size trace should produce a runtime point or sensitivity curve with the same qualitative behavior as the corresponding compact result.

The directory name is part of the trace provenance. In particular, the 128-GPU Llama directory identifies a 7B model. It must not be renamed to imply that it is a 70B trace. The exact vLLM 8B paper input is represented by the recovered local provenance manifest and compact results in A_1 ; the currently public vLLM 70B input is a different workload and must not be used as a substitute.

Expected Reproduction Time (in Minutes)

For one selected moderate trace:

- Artifact setup: 5–30 min to download the selected files, depending on network bandwidth.
- Artifact execution: about 5–45 min for conversion or replay.

- Artifact analysis: about 10 min to compare the generated runtime points with the compact reference.

Downloading the complete trace collection or processing Grok 4k is outside the recommended reviewer path.

Artifact Setup (incl. Inputs)

Hardware: Downloading and inspecting one selected trace requires a workstation with sufficient scratch space. Moderate GOAL replays need several GiB of RAM. Large Grok traces require cluster-scale scratch and memory and are not enabled by default.

Software: The downloader requires `wget` or `curl`. GOAL replay uses the LogGOPSim source included in A_1 . Raw NSYS conversion uses NVIDIA Nsight Systems and the NCCL converter in A_1 .

Datasets / Inputs: A_2 is the input dataset. The relevant public top-level directories are `llama/`, `grok/`, and `vllm/`. Each directory includes model and scale information in its name. The manifests in A_1 record the input identity and hashes used during the final reproduction work.

Installation and Deployment: Download only the trace needed for the selected evaluation. For example, the public 16-GPU Grok GOAL trace is under:

```
http://storage2.spcl.ethz.ch/traces/ai/grok/
Grok314B_N4_GPU16_TP4_PP1_CP1_VP1_
EP4_ETP4_GBS256/grok.goal
```

Place downloaded files in a scratch directory. Do not copy large raw traces into the Git checkout.

Artifact Execution

The data workflow is:

```
NSYS → SQLite → GOAL/BIN
→ LP or LogGOPSim → CSV → plot.
```

Prebuilt GOAL files enter at the third stage. The raw-input and GOAL workflows, their metadata requirements, and the validated exceptions are documented in A_1 .

Artifact Analysis (incl. Outputs)

The expected output is a GOAL replay runtime or sensitivity CSV. The evaluator should keep the exact source URL, file size, and hash with the output. The generated result can then be passed to the plotting stage in A_1 .

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Clone the artifact_freeze branch and install the Python requirements using the commands in the AD. From the repository root, first inspect the available work:

```
./reproduce_full.sh --dry-run
```

The dry run lists the enabled tasks without starting experiments. Confirm that data/, scripts/, reproduce_quick.sh, and reproduce_full.sh are present. Use scratch storage for all nontrivial runs.

For a local run, choose a scratch directory with:

```
mkdir -p /path/to/scratch/sc26-provisioning
```

On Alps, create the scratch directory first and use the Slurm mode described below. Do not launch trace conversion, LP sweeps, or LogGOPSim replays on a login node.

Artifact Execution

Quick evaluation. Run:

```
./reproduce_quick.sh
```

This validates the compact inputs and redraws Figures 1 and 3–10. The tasks are: load the committed CSV and JSON inputs, check their schemas, call the relevant plotting scripts, and check that the expected PDF and PNG outputs were created. No solver, large trace, GPU, or cluster allocation is used.

Full local evaluation without expensive stages. Run:

```
./reproduce_full.sh \
--scratch /path/to/scratch/sc26-provisioning \
--workers 4
```

The local driver runs the bounded tasks one after another. Solver workers within a task use the requested worker count. The core task redraws the figures and runs the manifest, artifact, and test checks. Each subsequent task writes result tables, comparison metrics, and a JSON execution manifest. It does not run Grok 4k or any stage marked as long or high-memory.

Full Slurm evaluation. On Alps or a similar cluster, run:

```
./reproduce_full.sh --slurm \
--scratch /path/to/scratch/sc26-provisioning
```

The driver submits experiments as separate batch jobs. Independent jobs run concurrently, while the two Figure 6 solver sweeps are serialized to respect Gurobi license limits. Each job writes its own log and JSON execution manifest. Monitor the submitted job IDs and inspect every manifest after completion. The core job performs the compact-data redraw. Inspect the generated job specifications first with `--dry-run`.

Opt-in expensive evaluation. The command below is documented for completeness and is not part of the recommended review:

```
./reproduce_full.sh --slurm --expensive_run \
--grok-analysis-dir /scratch/grok/N1024/analysis \
--scratch /large/scratch/sc26-provisioning
```

Use it only after confirming the time, memory, storage, and compute allocation. The flag enables Grok 4k and other stages that may exceed 45 min or require substantial memory.

Artifact Analysis (incl. Outputs)

After the quick run, confirm that plots for Figures 1 and 3–10 exist under figures/. After a full run, inspect the task log and the final report under docs/. Run the artifact checker documented by the top-level README and verify that it reports no missing required file or malformed compact dataset.

The expected comparisons are:

- Figures 1, 7, 8, 9, and 10 are redraws from existing data. Their plots should agree with the paper within plotting precision.
- Figures 3 and 4 should preserve the paper’s method ordering, sensitivity trends, and main breakpoints. Small configuration differences are listed in the report.
- Figure 5 uses the closest recovered raw-derived Composite result, whose maximum relative difference from the paper CSV is 0.014853%. The Monolithic and LGS series use existing compact data. Cold recovered-trace alternatives and their larger differences are documented in the report.
- Figure 6 should reproduce the plotted paper curves from compact data. Fresh Llama metadata analysis closely matches the packaged curve, subject to the 7B versus 70B label issue. The Grok 4k and vLLM paper curves use compact data by default.

For the Results Reproduced assessment, compare behavior rather than PDF pixels. Check the curve shapes, the ordering of the methods, the location of the major sensitivity transitions, and the provisioning conclusions. Use the supplied paper-versus-result comparison PDF and metrics tables for quantitative checks. The known differences must remain visible in the report.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Select one moderate trace rather than the full archive. The 16-GPU Grok GOAL trace is suitable for a functional check:

```
mkdir -p /path/to/scratch/sc26-trace
cd /path/to/scratch/sc26-trace
wget http://storage2.spcl.ethz.ch/traces/ai/grok/\
Grok314B_N4_GPU16_TP4_PP1_CP1_VP1_\
EP4_ETP4_GBS256/grok.goal
```

Return to the A_1 checkout and build LogGOPSim: `pipeline/build_tools.sh`

Artifact Execution

Replay the selected GOAL file at a representative latency:

```
python3 pipeline/run_lgs.py \  
--goal /path/to/scratch/sc26-trace/grok.goal \  
--L 4000
```

This path parses the GOAL schedule, converts it to the simulator's binary format if needed, and runs LogGOP-Sim. The output reports the predicted workload runtime. The full driver uses the same stages for its supported trace cases and stores reusable conversion files under the selected scratch directory.

Do not substitute the public vLLM 70B trace for the paper's vLLM 8B trace. Do not run the 4,096-GPU Grok trace during a routine evaluation.

Artifact Analysis (incl. Outputs)

Record the source URL, downloaded file size, hash, LogGOPSim parameters, and reported runtime. Compare the result with the matching compact reference or the tolerance stated in the final reproduction report. A successful parse and replay exercises the published trace, the simulator build, and the trace-to-runtime portion of the method. Use A_1 for the final plotting and paper-level comparison.